Eliminating Tool-in-the-Loop Debugging via Structural Verification for EDA Code Generation

Anonymous Authors

Abstract

Large language models (LLMs) have enabled natural-language-driven automation of electronic design automation (EDA) workflows, but reliable execution of generated scripts remains a fundamental challenge. In LLM-based EDA tasks, failures arise not from syntax errors but from violations of implicit structural dependencies over design objects, including invalid acquisition paths, missing prerequisites, and incompatible API usage. Existing approaches address these failures through tool-in-the-loop debugging, repeatedly executing and repairing programs using runtime feedback. While effective, this paradigm couples correctness to repeated tool invocation, leading to high latency and poor scalability in multi-step settings. We propose to eliminate tool-in-the-loop debugging by enforcing structural correctness prior to execution. Each task is represented as a structural dependency graph that serves as an explicit execution contract, and a verifier-guided synthesis framework enforces this contract through graph-conditioned retrieval, constrained generation, and staged pre-execution verification with diagnosis-driven repair. On single-step tasks, our method improves pass rate from 73.0% (LLM+RAG) and 76.0% (tool-in-loop) to 82.5%, while requiring exactly one tool call per task and reducing total tool calls by more than 2×. On multi-step tasks, pass rate improves from 30.0% to 70.0%, and further to 84.0% with trajectory-level reflection. Uncertainty-aware filtering further reduces verifier false positives from 20.0% to 6.7% and improves precision from 80.0% to 93.3%. These results show that enforcing structural consistency prior to execution decouples correctness from tool interaction, improving both reliability and efficiency in long-horizon EDA code generation.

1 Introduction

Recent advances in large language models (LLMs) have enabled the translation of natural-language intent into executable programs [Jiang et al.(2026)], opening the possibility of automating complex electronic design automation (EDA) workflows. In EDA, however, tasks require interacting with hierarchical design databases, composing tool-specific APIs, and maintaining consistency across tightly coupled flow stages. While natural-language-driven automation promises to reduce manual scripting effort, reliable execution of AI-generated EDA scripts remains a fundamental challenge.

The core difficulty arises from the structured and stateful nature of EDA environments. Unlike general-purpose code generation, correctness in EDA scripting depends on satisfying implicit dependencies over design objects and operations. Queries and transformations must follow valid object acquisition paths, respect type constraints, and satisfy action-specific preconditions defined by the design hierarchy. As a result, many generated programs fail not due to syntax errors, but due to structurally invalid execution paths, such as missing intermediate objects, invalid parent-child transitions, or incompatible API usage.

Existing LLM-based approaches address this problem primarily through *tool-in-the-loop debugging*, where candidate programs are generated, executed within the EDA tool, and iteratively repaired using runtime feedback. While effective, this paradigm treats the tool as a debugging oracle, leading to repeated execution, high latency, and poor scalability as task complexity increases. More importantly, it does not explicitly model the structural constraints that govern valid execution, relying instead on post-hoc feedback to detect failures. In multi-step query-action settings, where errors propagate across dependent operations, this results in inefficient trial-and-error loops and limited ability to localize root causes.

In this work, we argue that failures in EDA code generation are fundamentally *structural* rather than syntactic, and should be addressed *prior to execution*. Correct execution can be viewed as satisfying an implicit dependency graph over design objects and operations, which encodes valid acquisition paths, multi-branch dependencies, and action-specific constraints.

Based on this insight, we propose to eliminate tool-in-the-loop debugging by enforcing structural consistency before invoking the external tool. We introduce a framework that extracts a task-level *structural dependency graph* from the natural-language specification and uses it as an explicit execution contract. This graph guides retrieval, constrains code generation, and enables staged pre-execution verification. A multi-layer verifier detects violations such as missing dependencies, invalid object transitions, and API mismatches, while a controller performs diagnosis-driven, localized repair. By shifting error detection and correction to the pre-execution stage, the framework requires only a single tool invocation per task and avoids repeated EDA tool calls.

We evaluate the framework in OpenROAD on both single-step and multi-step query-action tasks over design databases. Compared to LLM+RAG baselines, the proposed method improves pass rate from 73.0% to 82.5% on single-step tasks and from 30.0% to 70.0% on multi-step tasks. Compared to tool-in-the-loop approaches, it achieves higher accuracy while reducing expensive OpenROAD calls by more than 2×, demonstrating that pre-execution structural verification improves both execution success and efficiency as task complexity increases.

Overall, the paper makes the following contributions:

- We identify structural dependency violations as a critical source of failure in EDA code generation and formalize execution correctness as consistency with a task-level dependency graph.
- We propose a pre-execution verification framework that eliminates reliance on tool-in-the-loop debugging by enforcing structural constraints before execution.
- We design a verifier with diagnosis-driven repair that localizes and corrects structural errors without repeated tool interaction.
- We demonstrate consistent improvements in correctness and efficiency across OpenROAD tasks, including multi-step workflows, multiple designs, and different PDKs.

2 Related Work

Agentic Code Generation. Recent advances in LLMs have improved code generation, enabling performance on multi-step reasoning, tool use, and long-horizon tasks. Frameworks such as ReAct [Yao et al.(2022)] and Reflexion [Shinn et al.(2023)] interleave reasoning and execution for iterative correction through execution traces or self-reflection. Related approaches, including Toolformer [Schick et al.(2023)], CodeAct [Wang et al.(2024)], program-aided reasoning [Gao et al.(2023)], and self-debugging [Chen et al.(2023)], show that structured feedback and tool integration improve correctness. More recent systems extend this paradigm with memory and planning for more complex multi-step behavior. Despite these advances, most approaches still generate candidate programs [Park et al.(2023)], evaluate them via external signals, and iteratively refine them. While effective when execution is cheap, this paradigm relies on post-hoc feedback rather than explicit structural modeling. In structured environments such as EDA, where correctness depends on multi-step dependencies, this leads to inefficient correction.

LLM-based EDA Scripting and OpenROAD. Recent work has explored LLMs for EDA automation, including script generation, design querying, and tool interaction. ChipNeMo [Liu et al.(2023)] and RTLCoder [Liu et al.(2024)] show that domain adaptation improves hardware design performance, especially for RTL generation and documentation grounding. EDA Corpus [Wu et al.(2024)] provides a public OpenROAD-oriented dataset for reproducible evaluation. Subsequent work focuses on OpenROAD , an open-source physical design system built on OpenDB with stateful stages such as placement, routing, and timing analysis [Ajayi and Blaauw(2019)]. Scripting requires navigating hierarchical design objects and composing tool-specific APIs across design states. OpenROAD-Assistant [Sharma et al.(2024)] and ORAssistant [Kaintura et al.(2024)] improve API grounding through retrieval-augmented generation but remain prompt-to-code assistants. OpenROAD-Agent adds execution-driven self-correction using runtime feedback, but relies on repeated tool invocation [Wu et al.(2025)]. In multi-step query-action tasks, this causes repeated execution and overhead as errors propagate across dependent operations.

Structure-Aware Verification. Program correctness in structured environments depends on dependency constraints between operations, not on syntax alone. Prior work in program synthesis uses structured representations [Osera and Zdancewic(2015)], such as abstract syntax trees, type systems, and syntax-guided synthesis, to constrain generation and reduce invalid outputs. Complementary approaches use generate-then-verify methods [Löding et al.(2016)], including type checking, constraint solving [Wang et al.(2018)], execution-guided filtering, and [Ni et al.(2023)] verifier-based selection . However, these methods generally focus on validating generic properties of generated programs, or depend on post-hoc execution feedback to detect errors, rather than enforcing the domain-specific dependency constraints that govern correct execution before the program is run. They do not capture task-level requirements such as valid object acquisition paths, multi-branch dependencies, and action-specific preconditions in EDA workflows. Feedback-driven methods such as Reflexion [Shinn et al.(2023)] and self-debugging improve correction, but still depend on global regeneration and repeated execution rather than explicitly isolating structural failures.

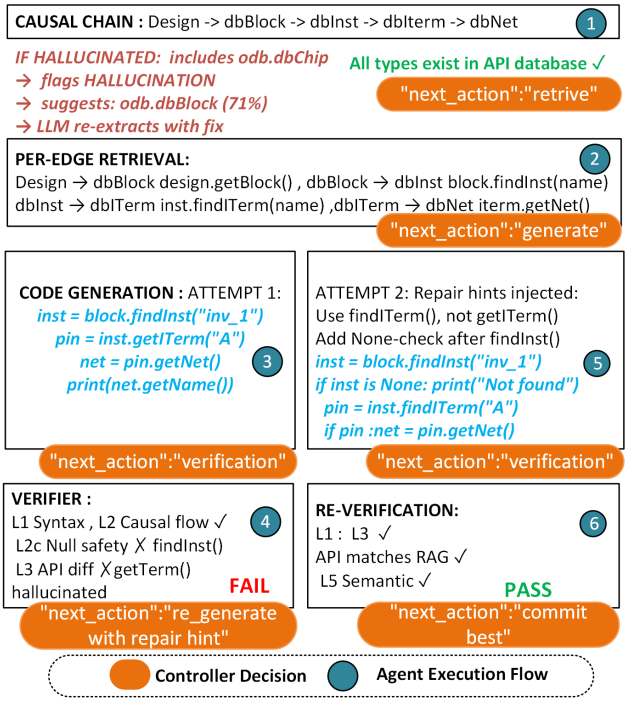

Figure 1: Verifier-guided structure-aware synthesis framework with staged verification and localized repair.

In contrast, our approach models EDA tasks as *structural dependency graphs* and enforces structural consistency before execution. Combined with graph-guided generation, staged verification, and localized repair, it avoids tool-in-the-loop debugging and improves efficiency and robustness.

3 Verifier-Guided Structure-Aware Synthesis

3.1 Methodology Overview

We address the problem of translating natural-language query-action prompts into executable OpenROAD programs over the OpenDB design database. Given a prompt P, the goal is to generate a program π that executes correctly in a structured, stateful physical design environment. Correctness is governed not only by syntax, but by *structural consistency* with the design hierarchy, including valid object acquisition paths, type compatibility, and action-specific preconditions. We therefore reformulate EDA code generation as enforcing structural consistency *prior to execution.*

In our framework, each task is modeled as a *structural dependency graph* $G = (V, E)$, where nodes represent typed design objects, intermediate conditions, and actions, and edges encode acquisition and dependency relations [Jayasuriya et al.(2025)]. This graph captures required object traversal paths and their composition, and serves as an explicit *execution contract.* A program is valid if it realizes the acquisition paths and satisfies the dependency constraints specified by G. Based on this formulation, we design a verifier-guided synthesis framework that enforces this contract before tool invocation.

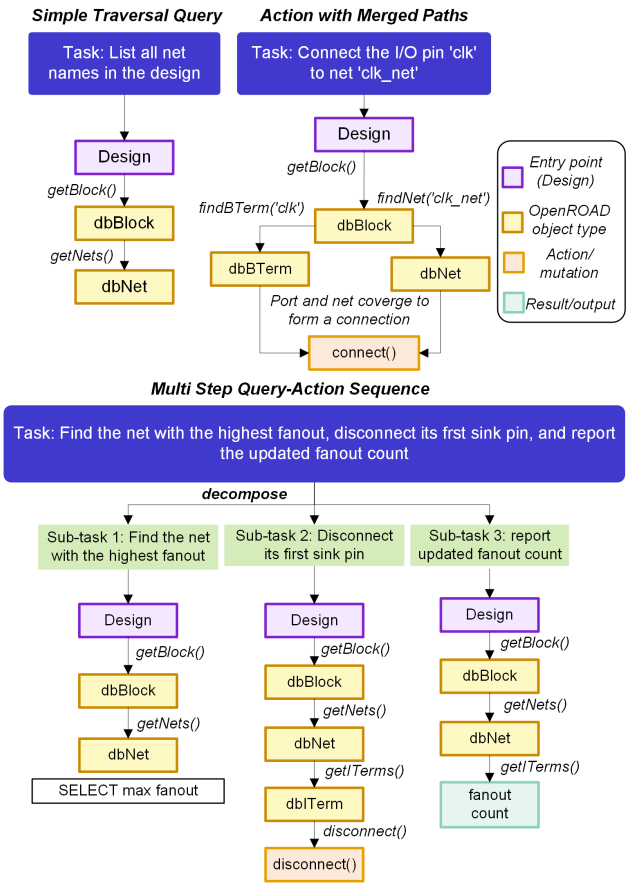

Figure 2: Structural representation of EDA query-action tasks as dependency graphs. Top: both simple traversal queries and action queries can be expressed as object-acquisition paths over the design hierarchy, with actions requiring the composition of multiple dependencies. Bottom: multi-step tasks decompose into a sequence of such graphs, where each step operates on the updated design state. This formulation exposes the structural constraints underlying execution and motivates enforcing consistency prior to tool invocation.

The framework operates in four stages: (1) graph construction from the prompt, (2) graph-conditioned retrieval and code generation, (3) staged pre-execution verification with diagnosis-driven repair, and (4) extension to multi-step execution with trajectory-level reflection. Unlike execution-driven approaches, all correction occurs prior to tool invocation, requiring a single OpenROAD execution per task. This eliminates reliance on tool-in-the-loop debugging, decouples repair from execution cost, and enables localized correction of structural errors rather than repeated trial-and-error interaction.

3.2 Structural Dependency Graph Construction

Direct prompt-to-code generation is insufficient because task structure is implicit in natural-language specifications. Even simple prompts require multiple dependent operations, including object acquisition, hierarchical traversal, condition evaluation, and action execution, which are not explicitly enumerated. As a result, correct execution depends on recovering the underlying dependency structure rather than generating code directly.

To make this structure explicit, we construct a *structural dependency graph* $G = (V, E)$. Nodes represent typed design objects, intermediate targets, conditions, and actions, while edges encode acquisition and dependency relations over the design hierarchy. This representation captures both required objects and their composition into valid execution paths.

The graph supports both linear and multi-branch dependencies. Linear chains correspond to hierarchical traversal (e.g., `Design` $\rightarrow$ `Block` $\rightarrow$ `Net`), while multi-branch structures capture independent object acquisition followed by convergence at an action node (e.g., `connect(Port, Net)`), avoiding spurious sequential dependencies.

We construct G via a hypothesis-and-validation process. An initial graph is proposed through structured LLM extraction, producing typed acquisition paths and an action node, which are deterministically parsed into graph form. The extracted graph is then validated against a structured OpenROAD API schema defining valid object types and permissible relations.

Each node and edge must satisfy type and relation constraints defined by the schema. Nodes are classified as valid, missing-but-real, or hallucinated, and edges are checked for valid parent-child transitions. Violations are converted into structured feedback and used to iteratively refine the graph until all constraints are satisfied. This step acts as a structural grounding gate, ensuring that only API-consistent graphs are passed to synthesis. By enforcing validity at the level of object dependencies rather than execution, the resulting graph serves as a reliable execution contract for downstream stages.

3.3 Structure-Grounded Synthesis

Given a validated structural dependency graph G, we perform structure-grounded synthesis, where both retrieval and generation are conditioned on the explicit dependency structure rather than the raw prompt. This shifts synthesis from unconstrained sequence generation to structured composition aligned with execution.

Graph-Conditioned Retrieval. Retrieval is decomposed into localized queries aligned with edges and substructures in G. Each query corresponds to an object transition or action dependency, retrieving API usage patterns and code fragments relevant to that relation. This yields evidence tied to individual dependencies, improving grounding while covering implicit prerequisite operations. Aligning retrieval with the graph reduces irrelevant context and helps recover missing intermediate steps.

Constrained Code Generation. Code generation is conditioned on the prompt, the dependency graph G, and the retrieved evidence. The graph enforces structural constraints, restricting generation to valid object-acquisition paths and dependency compositions, while retrieved evidence grounds API usage at each step. This reduces the search space by eliminating structurally invalid programs, such as those with missing prerequisites, incorrect traversal order, or incompatible object types. In contrast to prompt-only generation, where structure must be inferred, the graph provides an explicit plan that guides composition.

The resulting program aligns with the structural execution contract defined by G, but may still contain residual inconsistencies

in implementation or API usage. It is subsequently passed to the verifier-guided control stage for validation and targeted repair.

3.4 Verifier-Guided Control

This stage closes the synthesis loop by coupling generation with structured feedback and targeted repair, ensuring that the program satisfies the structural execution contract defined by G.

Four-Layer Verification. Each candidate is evaluated through a four-layer pipeline targeting progressively deeper error classes. We denote the failure layer as $\ell \in \{0, 1, 2, 3, 4\}$, where $\ell = 0$ indicates all layers are passed. Layer 1 (Syntax) ensures well-formed Python via AST parsing. Layers 2 and 3 perform **causal verification** against G. Layer 2 (Causal Flow) verifies correct acquisition order, valid attribute access, and proper handling of nullable returns. Layer 3 (API Alignment) checks that invoked methods match retrieved APIs, detecting hallucinated calls and incorrect object usage. Layer 4 (Semantic) evaluates task-level consistency using an LLM judge, including completeness, control flow, and output validity, and is applied only after L1–L3 pass.

Diagnosis-Driven Repair. The verifier produces a structured diagnostic report specifying failure stage, issues, and localized inconsistencies (e.g., missing acquisition steps, invalid transitions, receiver mismatches, incorrect APIs). Each failure is mapped to a region of G, enabling localized correction rather than global regeneration. A controller selects actions based on the task, graph, retrieved APIs, prior actions, feedback, and remaining budget: **Regeneration** (regenerate code with targeted repair hints for L1, L2, or persistent L3 failures), **Edge re-retrieval** (update retrieval for a specific graph edge and regenerate for L3 failures), **Graph re-extraction** (revise G and restart synthesis for repeated L2 failures), and **Accept** (commit when minor issues remain but the program is semantically valid). A deterministic safeguard prevents repeated ineffective actions by detecting loops and escalating to alternative strategies.

Execution Efficiency. All correction occurs prior to tool invocation, requiring exactly one OpenROAD call per task. This decouples repair from execution cost and avoids repeated tool interaction. Increasing the repair budget therefore improves solution quality without increasing tool usage, unlike execution-driven approaches where each iteration incurs an additional call.

3.5 Post-Verification Uncertainty Estimation

The verifier enforces structural correctness but produces a binary decision. In practice, some verifier-Pass programs still fail in OpenROAD due to unsupported API usage or unstable repair trajectories. This highlights a limitation of purely structural verification: passing the verifier is necessary but not always sufficient for reliable execution [Kumar et al.(2025)]. We therefore introduce a post-verification uncertainty estimate that scores the reliability of verifier-Pass programs prior to execution [Tayebati et al.(2026)]. We decompose uncertainty into three sources: *code-level uncertainty*, capturing suspicious API usage in the final program; *trajectory-level uncertainty*, capturing instability in the repair process; and *verification-coverage uncertainty*, capturing how much of the program is actually validated. Let $\mathbf{x}$ denote the final program and $\mathcal{T}$ its repair trajectory: $u(\mathbf{x}, \mathcal{T}) = \alpha_1 u_{\text{code}}(\mathbf{x}) + \alpha_2 u_{\text{traj}}(\mathbf{x}) + \alpha_3 u_{\text{cov}}(\mathcal{T})$, where $\alpha_1 + \alpha_2 + \alpha_3 = 1$.

3.5.1 Code-Level Uncertainty. Code-level uncertainty targets residual failures where structurally valid programs reference unsupported API constructs. Let $\mathcal{A}_m$ and $\mathcal{A}_t$ denote valid method and type sets. We define three hallucination signals:

$$\begin{aligned} h_m(\mathbf{x}) \ \textbf{(method hallucination)} &= \frac{|\{m \in M(\mathbf{x}) : m \notin \mathcal{A}_m\}|}{|M(\mathbf{x})|}, \\ h_I(\mathbf{x}) \ \textbf{(invalid imports)} &= |\{i \in I(\mathbf{x}) : i \notin \mathcal{A}_m \cup \mathcal{A}_t\}|, \\ h_E(\mathbf{x}) \ \textbf{(enum hallucination)} &= |\{e \in E(\mathbf{x}) : e \notin \mathcal{E}_{\text{known}}\}|. \end{aligned} \tag{1}$$

These are combined into a confidence score

$$c_{\text{code}}(\mathbf{x}) = \text{clip}[1 - \lambda_I h_I(\mathbf{x}) - \lambda_E h_E(\mathbf{x}) - \lambda_r h_m(\mathbf{x}),\ 0,\ 1], \tag{2}$$

and uncertainty is defined as $u_{\text{code}}(\mathbf{x}) = 1 - c_{\text{code}}(\mathbf{x})$.

3.5.2 Trajectory-Level Uncertainty. Trajectory-level uncertainty captures instability in the repair process. Let $\ell_0, \ldots, \ell_T$ denote verifier outcomes ($\ell_t \in \{0, \ldots, 4\}$) and $\mathbf{x}_0, \ldots, \mathbf{x}_T$ the candidates.

$$\begin{aligned} \tau_{\text{conv}} \ \textbf{(convergence)} &= \max\left(0,\ 1 - \frac{\ell_0 - \ell_T}{\max(\ell_0, 1)}\right), \\ \tau_{\text{stag}} \ \textbf{(stagnation)} &= \frac{1}{T}\sum_{t=0}^{T-1} J(\mathbf{x}_t, \mathbf{x}_{t+1}), \\ \tau_{\text{eff}} \ \textbf{(repair ineffectiveness)} &= \frac{|\{t \in R : \ell_t \geq \ell_{t-1}\}|}{|R|}. \end{aligned} \tag{3}$$

We combine these as

$$u_{\text{traj}}(\mathcal{T}) = w_c \tau_{\text{conv}} + w_s \tau_{\text{stag}} + w_e \tau_{\text{eff}}. \tag{4}$$

3.5.3 Verification-Coverage Uncertainty. Verification-coverage uncertainty captures cases where a verifier Pass reflects incomplete checking rather than confirmed correctness. Let $\mathcal{K}(\tau)$ denote the set of known methods for receiver type τ, and let $C(\mathbf{x})$ be the set of method calls in program $\mathbf{x}$, with receiver types inferred via lightweight propagation. We define two coverage signals:

$$\begin{aligned} c_{\text{cov}}(\mathbf{x}) \ \textbf{(verified-call coverage)} &= \frac{|C_{\text{cov}}(\mathbf{x})|}{|C(\mathbf{x})|}, \\ u_{\text{unc}}(\mathbf{x}) \ \textbf{(unchecked-call ratio)} &= \frac{|C_{\text{unc}}(\mathbf{x})|}{|C(\mathbf{x})|}, \end{aligned} \tag{5}$$

where

$$\begin{aligned} C_{\text{cov}}(\mathbf{x}) &= \{(\tau, m) \in C(\mathbf{x}) : \tau \text{ is well-covered and } m \in \mathcal{K}(\tau)\}, \\ C_{\text{unc}}(\mathbf{x}) &= C(\mathbf{x}) \setminus C_{\text{cov}}(\mathbf{x}). \end{aligned} \tag{6}$$

Since these quantities are complementary, we define

$$u_{\text{cov}}(\mathbf{x}) = 1 - c_{\text{cov}}(\mathbf{x}) = \frac{|C_{\text{unc}}(\mathbf{x})|}{|C(\mathbf{x})|}. \tag{7}$$

u_{code}, u_{traj}, and u_{cov} capture complementary risks where unsupported API behavior, unverifiable API usage, and weak convergence, providing a more informative estimate than the verifier's binary verdict and enabling uncertainty-aware filtering prior to execution.

Table 1: Single-step query-action performance and efficiency. Higher pass rate, lower latency and fewer tool calls are better.

Method	Pass Rate (%)	Latency (s)	Calls / Task	Total Calls
Static verifier only	50.8	19.0	1.00	120
Causal verifier only	79.2	27.5	1.00	120
Full pipeline	**82.5**	34.8	**1.00**	**120**
Tool-in-loop (GPT-4.1-mini)	76.0	53.0	1.77	248
OpenROAD Agent (Qwen 7B)	16.4	70.0	3.54	496

3.6 Extension to Multi-Step Tasks

We extend the framework to multi-step query-action prompts by decomposing each input into an ordered sequence of subtasks. Each subtask follows the same pipeline, subtask → graph → validation → retrieval → generation → verification, and is executed only after successful verification. Accepted programs update the design state, which is propagated to subsequent steps.

A key challenge is cascading failure, where errors in early steps corrupt the design state and cause downstream failures, even when later steps are locally correct. Thus, correctness must be enforced both locally and across the full execution sequence. If a subtask fails after exhausting the repair budget, execution is terminated to prevent propagation of corrupted state.

To address this, we introduce an episode-level reflection mechanism for cross-step failure diagnosis. After an initial pass, the framework analyzes the full sequence of subtask outcomes to identify root causes of failure. Reflection captures cross-step dependencies and error propagation not observable within individual subtasks, including inconsistencies between earlier object acquisition and later usage. The resulting diagnostic signal guides a second-pass synthesis with targeted corrections, enabling recovery from failures that cannot be resolved through local verification alone. This pass corrects upstream structural errors rather than repeatedly repairing downstream symptoms [Madaan et al.(2023)].

4 Results

4.1 Benchmark and Evaluation Protocol

We evaluate the proposed framework on generating executable OpenROAD programs from natural-language query-action prompts over the OpenDB design database. The evaluation focuses on (i) execution correctness and (ii) efficiency in terms of OpenROAD tool usage. We consider two task settings. *First*, single-step query-action prompts, where each prompt specifies a compact OpenROAD operation. These provide a controlled setting for evaluating generation quality and verifier effectiveness, although correct execution typically requires multiple dependent API calls over the design hierarchy. *Second*, multi-step query-action prompts, where each prompt consists of a sequence of dependent subtasks executed within a single OpenROAD session. This setting is more challenging, as correctness depends on both step-level validity and consistency of the evolving design state. Since no standard benchmark exists, we construct a dataset of 100 complex prompts via adversarial self-play [Lin et al.(2025)], covering diverse object hierarchies, API interactions, and multi-stage operations [Cheng et al.(2024)].

A task is considered successful if the generated code executes in the OpenROAD Python shell without raising an exception or traceback. Execution is performed in a live OpenROAD process via a pseudo-terminal. A task is marked as failed if a traceback occurs, if the code contains a syntax error, if execution exceeds 120 seconds, or if the process crashes. While execution success does not guarantee semantic optimality, it is a necessary condition for usable EDA scripts. In OpenROAD, incorrect API usage typically manifests as runtime exceptions, making execution success a strong proxy for correctness. Structural pre-execution verification and post-verification uncertainty filtering mitigate residual risks.

For multi-step tasks, subtasks are executed sequentially within a single session, preserving verified state across steps. A task is successful only if all subtasks execute successfully; any failure terminates execution to prevent error propagation. In addition to correctness, we measure efficiency via OpenROAD tool calls and end-to-end latency, enabling direct comparison with execution-driven approaches where each repair iteration incurs an additional tool call. We also evaluate generalization across designs, PDKs, and OpenROAD stages, e.g. floorplanning and placement.

4.2 Key Results on Single-Step Tasks

We evaluate the proposed framework on single-step query-action prompts, a controlled setting for assessing execution correctness. Although each prompt specifies a single operation, correct execution typically requires multiple dependent API calls over the OpenDB hierarchy. Table 1 summarizes the results. The full pipeline achieves a pass rate of **82.5%**, outperforming LLM+RAG (73.0%) and tool-in-the-loop correction (76.0%), showing that enforcing structural correctness prior to execution improves reliability [Lewis et al.(2020)].

To isolate component contributions, we evaluate ablated variants. The static verifier achieves 50.8% pass rate, while the causal verifier improves performance to 79.2%, indicating that enforcing object-acquisition structure provides the dominant gain. The full pipeline further increases performance to 82.5%, showing that semantic validation and repair provide smaller, incremental improvements. Compared to prior systems, the gains are substantial: OpenROAD Agent achieves 16.4% pass rate, and only 31.8% even with a verifier, highlighting the importance of integrating structural verification during generation rather than applying it post hoc. Table 2 shows consistent trends across models. For GPT-4.1-mini, retrieval improves pass rate from 48.0% to 73.0%, and tool-in-the-loop correction to 76.0%, while our method reaches 82.5%. Similar gains hold for GPT-4o (52% to 83%), demonstrating robustness across models. These results indicate that retrieval alone is insufficient, as it does not enforce structural consistency required for correct execution.

4.3 Efficiency and Tool Call Reduction

Table 1 shows that the proposed framework requires exactly **one OpenROAD call per task**, compared to 1.77 for tool-in-the-loop correction and 3.54 for OpenROAD Agent. Across the evaluation set, this corresponds to **120 total calls** versus 248 and 496, yielding more than a 2× reduction in tool usage. Latency follows a similar trend. By performing all correction prior to invocation, the framework executes each task only once. Although the full pipeline introduces modest overhead from verification and repair (34.8s vs

Table 2: OpenROAD script generation performance across systems. We report pass rate, tool usage, and latency.

Model	Method	Generator	Verifier	RAG	Pass (%)	Calls / Task	Latency (s)
Qwen2.5-7B	OpenROAD-Agent	Qwen2.5-ft	—	✓	16.4	3.54	69.5
Qwen2.5-7B	OpenROAD-Agent + verifier	Qwen2.5-ft	GPT-4.1-mini	✓	31.8	1.00	33.4
GPT-4.1-mini	LLM only	GPT-4.1-mini	—	×	48.0	—	—
GPT-4.1-mini	LLM + RAG	GPT-4.1-mini	—	✓	73.0	—	—
GPT-4.1-mini	LLM + tool-in-loop	GPT-4.1-mini	—	✓	76.0	1.77	37.6
GPT-4.1-mini	**Ours (full)**	GPT-4.1-mini	GPT-4.1-mini	✓	**82.5**	**1.00**	34.8
GPT-4o	LLM only	GPT-4o	—	×	52.0	—	—
GPT-4o	**Ours (full)**	GPT-4o	GPT-4o	✓	**83.0**	1.00	36.2

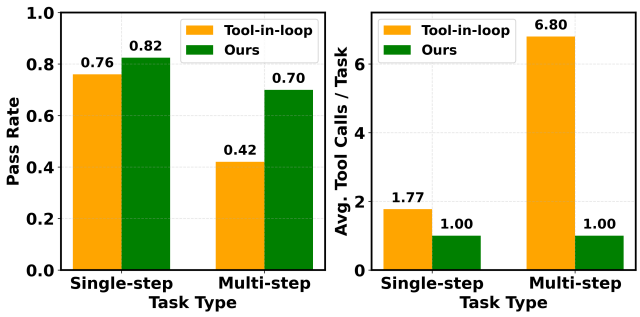

Figure 3: Pass rate and tool usage comparison across single-step and multi-step tasks. The proposed framework maintains higher accuracy while requiring a constant single tool call per task, whereas tool-in-the-loop methods incur significantly higher tool usage and degrade in multi-step settings.

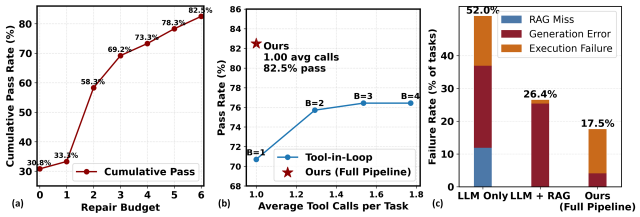

Figure 4: Effect of repair budget and tool interaction on execution performance. (a) Cumulative pass rate improves with repair budget, reflecting gains from iterative refinement. (b) Pass rate versus average tool calls per task: the proposed framework achieves higher accuracy (82.5%) with a single tool call, while tool-in-the-loop methods require multiple executions for smaller gains. (c) Failure mode breakdown highlighting reductions in generation and execution errors.

Table 3: Multi-step query–action performance.

Method	Verifier	RAG	Pass (%)	Latency (s)
LLM + RAG	—	✓	30.0	12.2
Ours	✓	✓	70.0	53.9
Ours + Reflection	✓	✓	**84.0**	~94.9

27.5s for causal-only), it remains significantly faster than tool-in-the-loop correction (53.0s) and OpenROAD Agent (70s).

Fig. 3 compares pass rate and tool usage across single-step and multi-step settings. The proposed method maintains higher accuracy with a constant single tool call per task, while tool-in-the-loop approaches degrade in multi-step settings and incur substantially higher tool usage (up to 6.8 calls per task). This shows that execution cost scales with task complexity for tool-in-the-loop methods but remains fixed under pre-execution verification. Fig. 4 further shows that increasing the repair budget improves pass rate without increasing tool usage. In contrast, tool-in-the-loop methods incur additional tool calls with each repair iteration.

4.4 Multi-Step Execution Performance

We evaluate the proposed framework on multi-step query-action prompts, where each task consists of dependent subtasks executed within a single OpenROAD session. This setting is substantially more challenging than single-step tasks, as correctness depends on both step-level validity and consistency of the evolving design state. Since no standard benchmark exists for this regime, we construct a multi-step dataset through round-based adversarial evolution, where a creator model increases prompt complexity and a solver model improves solvability across rounds. Figure 6 summarizes this process. Compared to base EDA-Corpus tasks, the evolved prompts exhibit greater structural complexity and expose a broader range of failure types, including API mismatches, missing prerequisites, type inconsistencies, and semantic failures. Fig. 6(c) further shows that new failure modes continue to emerge across rounds, indicating increasing task diversity rather than overfitting to a fixed error distribution.

Table 3 summarizes multi-step execution results on this benchmark. LLM+RAG achieves a pass rate of 30.0%, reflecting failures due to missing dependencies and inconsistent state propagation. Enforcing structural correctness at each step improves performance to 70.0%, indicating that most local errors can be resolved through per-step verification even as task complexity increases.Incorporating trajectory-level reflection further improves pass rate from 70.0% to 84.0%. This shows that many of the remaining failures arise not from local step errors, but from inconsistencies that propagate across steps. By reasoning over the full trajectory, reflection helps identify and correct these upstream errors.

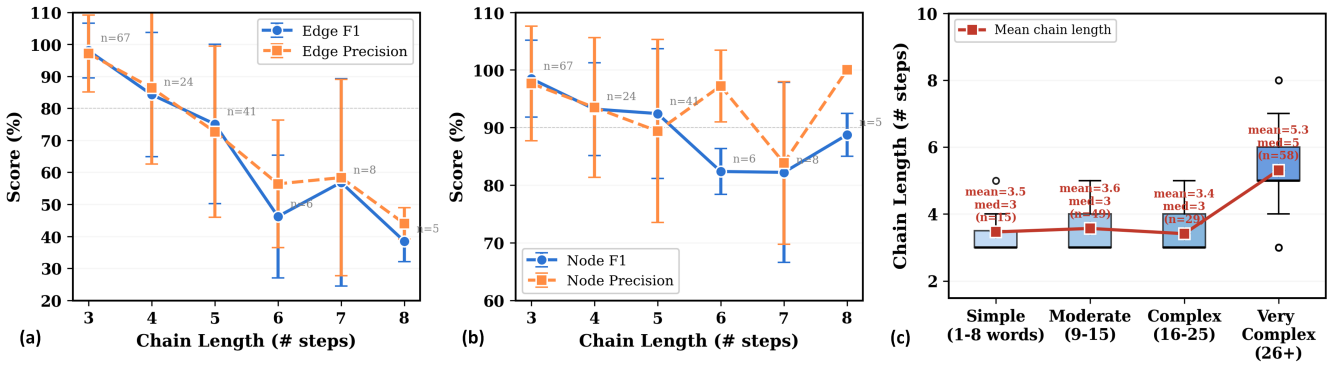

Figure 5: Causal-graph extraction accuracy. (a) Edge-level F1 and precision as a function of chain length. (b) Node-level F1 and precision as a function of chain length. (c) Ground-truth chain length across prompt complexity buckets.

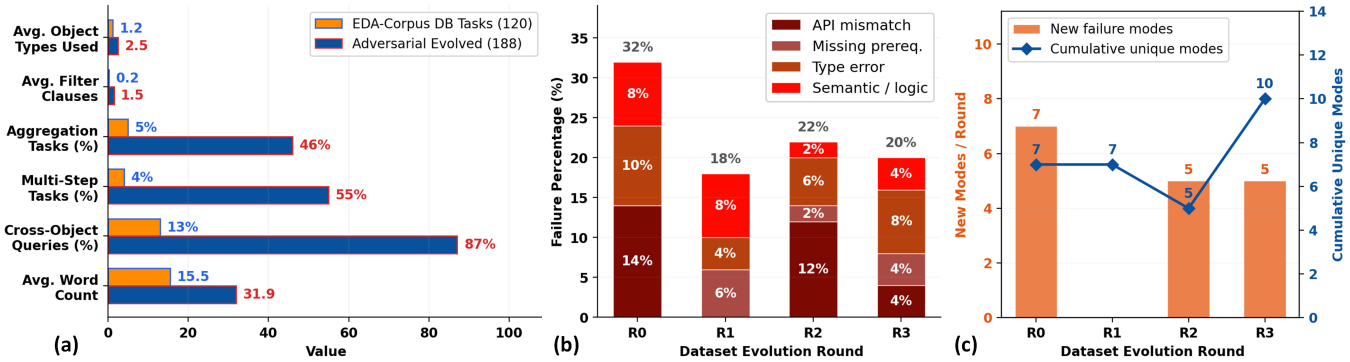

Figure 6: Round-based adversarial evolution for multi-step benchmark construction. (a) Compared to base EDA-Corpus tasks, evolved prompts exhibit greater structural complexity, including multi-step composition, aggregation, and cross-object interactions. (b) Failure types observed across evolution rounds, including API mismatches, missing prerequisites, type inconsistencies, and semantic errors. (c) Newly discovered and cumulative failure modes across rounds, showing that later rounds introduce more diverse and challenging tasks rather than repeating a fixed error distribution.

Table 4: Causal-chain accuracy by prompt complexity

Complexity Level (<word count)	Node F1 (%)	Edge F1 (%)	Exact Match (%)
Simple (<8)	100	100	100
Moderate (<15)	99	93	96
Complex (<25)	96	88	76
Very Complex (26+)	87	69	31

4.5 Ablation Studies on Structural Components

We analyze the contribution of structural graph extraction and staged verification to explain the observed performance gains.

4.5.1 Structural Graph Extraction Accuracy. We evaluate the quality of extracted structural dependency graphs by comparing them against ground-truth acquisition graphs recovered from passing programs via static AST analysis. Node and edge correspondences are resolved using an OpenDB API return-type table.

As shown in Fig. 5, the extracted graphs achieve strong node-level accuracy (F1 = 0.869), reliably recovering required object types. Recall (0.911) exceeds precision (0.861), indicating a tendency to add extra nodes rather than miss necessary ones. Edge-level accuracy is lower but still substantial (F1 = 0.680), and exact graph match is achieved in 25% of cases [Sokolova and Lapalme(2009)] .

Table 4 summarizes accuracy across prompt complexity. While performance remains near-perfect for simple tasks, exact match [Stengel-Eskin and Van Durme(2023)] drops to 31% for very complex prompts, confirming that precise structural recovery becomes increasingly difficult with complexity. Despite this, execution is not significantly impacted, as extra nodes improve retrieval coverage and missing dependencies are corrected during verification. Thus, approximate structural extraction provides a sufficient prior for correct execution .

4.5.2 Verifier Contribution and Staged Enforcement. Staged verification improves execution reliability mainly through structural constraint enforcement. Causal verification raises precision from 0.508 to 0.807 and F1 from 0.674 to 0.880 over static checking. Semantic checking then refines borderline cases, and the full staged verifier achieves the best overall precision-recall balance.

Table 5: Verifier quality on single-task prompts.

Verifier Stage	Precision	Recall	F1
Static Verifier	0.508	1.000	0.674
Causal Verifier	0.807	0.968	0.880
LLM Semantic Verifier	0.849	1.000	**0.918**
Full Staged Verifier	0.849	0.909	0.878

Table 6: Effect of uncertainty-aware filtering on pass rate and verifier precision.

Metric	Baseline	+ Uncertainty	Δ
Step 1 pass rate	80.0%	92.0%	+12.0%
Step 2 pass rate	78.0%	90.0%	+12.0%
Overall pass rate	72.0%	84.0%	+12.0%
Verifier false positive rate	20.0%	6.7%	−13.3%
Verifier precision	80.0%	93.3%	+13.3%

Table 7: Cross-design and cross-PDK (Process Design Kit) generalization of the proposed framework. Inst: standard-cell instances; Nets: signal nets; IO: input/output pins; Lat (s): average task latency; LLM: average large language model calls per task; Cost: estimated cost per task using GPT-4.1-mini pricing ($0.40/M input tokens, $1.60/M output tokens).

Design	PDK	Node	Inst	Nets	IO	Pass	Lat (s)	LLM	Cost ($)
GCD	Nangate45	45nm	624	581	54	81.5%	28.0	3.0	0.0028
GCD	sky130hd	130nm	1,143	306	54	88.9%	20.0	2.2	0.0021
GCD	sky130hs	130nm	366	402	54	94.4%	24.1	2.4	0.0019
AES	Nangate45	45nm	156,172	17,388	388	88.9%	51.5	2.1	0.0020
RISC-V	ASAP7	7nm	12,161	9,585	135	90.7%	22.5	1.26	0.0017
AES	ASAP7	7nm	22,107	21,542	388	94.4%	21.6	1.04	0.0013

4.6 Uncertainty and Reliability Analysis

While staged verification substantially improves execution correctness (Section 4.2), residual failures arise when verification is incomplete or repair does not converge. We therefore introduce uncertainty measures to estimate reliability prior to execution and enable selective filtering of low-confidence programs. Table 6 summarizes the impact of uncertainty-aware filtering. Without filtering, the verifier exhibits a false positive rate of 20.0% and precision of 80.0%, indicating that a non-trivial fraction of programs marked as valid still fail at execution. Incorporating uncertainty reduces the false positive rate to **6.7%** and improves precision to **93.3%**, showing that uncertainty signals effectively identify unreliable candidates.

As explained in the section 3.5 we consider three complementary sources of uncertainty [Wang et al.(2022)]. These signals provide complementary views of failure: code-level uncertainty targets local inconsistencies, trajectory-level uncertainty reflects weak convergence, and coverage uncertainty captures blind spots in verification . Combining them yields a more informative reliability estimate than the verifier's binary decision [Geifman and El-Yaniv(2017)].

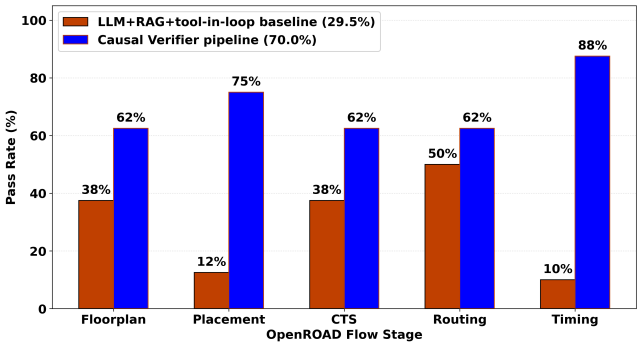

Figure 7: Stage-wise performance across OpenROAD flow stages, showing consistent pass rates across diverse operations and API patterns.

4.7 Generalization Across Designs and PDKs

We evaluate the proposed framework across different designs, process design kits (PDKs), and OpenROAD flow stages. Unlike prompt-level approaches that may overfit to specific API patterns, structural verification operates on dependency constraints, enabling transfer across diverse configurations.

Table 7 summarizes performance across designs and PDKs. The framework maintains high pass rates, indicating that structural consistency generalizes beyond prompt-construction distributions. Performance remains stable across variations in object hierarchies, API usage patterns, and flow stages. Fig. 7 shows stage-wise results across OpenROAD tasks, including floorplanning, placement, CTS, routing, and timing. The method achieves consistent accuracy across stages, demonstrating robustness to diverse operations and API compositions. In contrast, baseline approaches show higher variability. Retrieval-based methods are sensitive to shifts in API usage, while tool-in-the-loop approaches incur higher cost from repeated execution, especially when errors propagate across stage-specific dependencies. These results suggest that structural dependency modeling captures invariant properties of EDA workflows, improving robustness while maintaining low tool usage.

5 Conclusion

We presented a framework that eliminates tool-in-the-loop debugging in LLM-based EDA code generation by enforcing structural correctness before execution. By modeling tasks as structural dependency graphs used as execution contracts, the approach shifts correctness from post-hoc feedback to pre-execution verification, enabling graph-conditioned synthesis, staged verification, and diagnosis-driven repair with a single tool call per task. Experiments in OpenROAD show improved reliability and efficiency. The method reaches 82.5% single-step pass rate while reducing tool calls by more than 2×, and scales to multi-step tasks, achieving up to 84.0% with trajectory-level reflection. Uncertainty-aware filtering further improves reliability, and results across designs, PDKs, and flow stages show strong generalization. Overall, the results suggest that failures in EDA code generation are fundamentally structural, and that enforcing dependency consistency before execution offers a scalable alternative to execution-driven correction.

References

[Ajayi and Blaauw(2019)] Tutu Ajayi and David Blaauw. 2019. Openroad: Toward a self-driving, open-source digital layout implementation tool chain. In *Proceedings of Government Microcircuit Applications and Critical Technology Conference*.

[Chen et al.(2023)] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2023. Teaching large language models to self-debug. *arXiv preprint arXiv:2304.05128* (2023).

[Cheng et al.(2024)] Pengyu Cheng, Yong Dai, Tianhao Hu, Han Xu, Zhisong Zhang, Lei Han, Nan Du, and Xiaolong Li. 2024. Self-playing adversarial language game enhances llm reasoning. *Advances in Neural Information Processing Systems* 37 (2024), 126515–126543.

[Gao et al.(2023)] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. Pal: Program-aided language models. In *International conference on machine learning*. PMLR, 10764–10799.

[Geifman and El-Yaniv(2017)] Yonatan Geifman and Ran El-Yaniv. 2017. Selective classification for deep neural networks. *Advances in neural information processing systems* 30 (2017).

[Jayasuriya et al.(2025)] Dinithi Jayasuriya, Divake Kumar, Sureshkumar Senthilkumar, Devashri Naik, Nastaran Darabi, and Amit Ranjan Trivedi. 2025. Causal-Guided Dimension Reduction for Efficient Pareto Optimization. *arXiv preprint arXiv:2510.09941* (2025).

[Jiang et al.(2026)] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2026. A survey on large language models for code generation. *ACM Transactions on Software Engineering and Methodology* 35, 2 (2026), 1–72.

[Kaintura et al.(2024)] Aviral Kaintura, Shui Song Luar, Indira Iyer Almeida, et al. 2024. ORAssistant: A custom RAG-based conversational assistant for OpenROAD. *arXiv preprint arXiv:2410.03845* (2024).

[Kumar et al.(2025)] Divake Kumar, Patrick Poggi, Sina Tayebati, Devashri Naik, Nilesh Ahuja, and Amit Ranjan Trivedi. 2025. Calibrated Decomposition of Aleatoric and Epistemic Uncertainty in Deep Features for Inference-Time Adaptation. *arXiv preprint arXiv:2511.12389* (2025).

[Lewis et al.(2020)] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.

[Lin et al.(2025)] Zi Lin, Sheng Shen, Ilia Kulikov, Jingbo Shang, Jason Weston, and Yixin Nie. 2025. Learning to solve and verify: A self-play framework for code and test generation. *arXiv preprint arXiv:2502.14948* (2025).

[Liu et al.(2023)] Mingjie Liu, Teodor-Dumitru Ene, Robert Kirby, Chris Cheng, Nathaniel Pinckney, Rongjian Liang, Jonah Alben, Himyanshu Anand, Sanmitra Banerjee, Ismet Bayraktaroglu, et al. 2023. Chipnemo: Domain-adapted llms for chip design. *arXiv preprint arXiv:2311.00176* (2023).

[Liu et al.(2024)] Shang Liu, Wenji Fang, Yao Lu, Qijun Zhang, Hongce Zhang, and Zhiyao Xie. 2024. Rtlcoder: Outperforming gpt-3.5 in design rtl generation with our open-source dataset and lightweight solution. In *2024 IEEE LLM Aided Design Workshop (LAD)*. IEEE, 1–5.

[Löding et al.(2016)] Christof Löding, P Madhusudan, and Daniel Neider. 2016. Abstract learning frameworks for synthesis. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 167–185.

[Madaan et al.(2023)] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. 2023. Self-refine: Iterative refinement with self-feedback. *Advances in neural information processing systems* 36 (2023), 46534–46594.

[Ni et al.(2023)] Ansong Ni, Srini Iyer, Dragomir Radev, Veselin Stoyanov, Wen-tau Yih, Sida Wang, and Xi Victoria Lin. 2023. Lever: Learning to verify language-to-code generation with execution. In *International Conference on Machine Learning*. PMLR, 26106–26128.

[Osera and Zdancewic(2015)] Peter-Michael Osera and Steve Zdancewic. 2015. Type-and-example-directed program synthesis. *ACM SIGPLAN Notices* 50, 6 (2015), 619–630.

[Park et al.(2023)] Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. 2023. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*. 1–22.

[Schick et al.(2023)] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems* 36 (2023), 68539–68551.

[Sharma et al.(2024)] Utsav Sharma, Bing-Yue Wu, Sai Rahul Dhanvi Kankipati, Vidya A Chhabria, and Austin Rovinski. 2024. Openroad-assistant: An open-source large language model for physical design tasks. In *Proceedings of the 2024 ACM/IEEE International Symposium on Machine Learning for CAD*. 1–7.

[Shinn et al.(2023)] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems* 36 (2023), 8634–8652.

[Sokolova and Lapalme(2009)] Marina Sokolova and Guy Lapalme. 2009. A systematic analysis of performance measures for classification tasks. *Information processing & management* 45, 4 (2009), 427–437.

[Stengel-Eskin and Van Durme(2023)] Elias Stengel-Eskin and Benjamin Van Durme. 2023. Calibrated interpretation: Confidence estimation in semantic parsing. *Transactions of the Association for Computational Linguistics* 11 (2023), 1213–1231.

[Tayebati et al.(2026)] Sina Tayebati, Divake Kumar, Nastaran Darabi, Davide Ettori, Ranganath Krishnan, and Amit Ranjan Trivedi. 2026. TRACER: Trajectory Risk Aggregation for Critical Episodes in Agentic Reasoning. *arXiv preprint arXiv:2602.11409* (2026).

[Wang et al.(2018)] Chenglong Wang, Kedar Tatwawadi, Marc Brockschmidt, Po-Sen Huang, Yi Mao, Oleksandr Polozov, and Rishabh Singh. 2018. Robust text-to-sql generation with execution-guided decoding. *arXiv preprint arXiv:1807.03100* (2018).

[Wang et al.(2024)] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. 2024. Executable code actions elicit better llm agents, 2024. *URL https://arxiv. org/abs/2402.01030* 6, 1 (2024), 6–2.

[Wang et al.(2022)] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2022. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171* (2022).

[Wu et al.(2024)] Bing-Yue Wu, Utsav Sharma, Sai Rahul Dhanvi Kankipati, Ajay Yadav, Bintu Kappil George, Sai Ritish Guntupalli, Austin Rovinski, and Vidya A Chhabria. 2024. Eda corpus: A large language model dataset for enhanced interaction with openroad. In *2024 IEEE LLM Aided Design Workshop (LAD)*. IEEE, 1–5.

[Wu et al.(2025)] Bing-Yue Wu, Utsav Sharma, Austin Rovinski, and Vidya A Chhabria. 2025. OpenROAD Agent: An Intelligent Self-Correcting Script Generator for OpenROAD. In *2025 IEEE International Conference on LLM-Aided Design (ICLAD)*. IEEE, 16–22.

[Yao et al.(2022)] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*.